Neural Ledger

# Security Self-Assessment Report

Controls, evidence, known risks and residual risk after option B

An AI cash-flow copilot with deterministic authority
FinTechathon 2026 · International Track (Agent Wallet) · Topic A: Personal Finance Assistant

| Role | Owner |
|---|---|
| A · Finance core | Bruno Caro |
| B · Agent / AI | Luis-Felipe |
| C · Platform | Baptiste |
| D · Frontend | Vicente Ramírez |
| E · Quality and submission | Joaquín Parraud |

| | |
|---|---|
| Date | 20 October 2026 |
| Document version | 1.2 |
| Software version | 0.1.0 (`backend/pyproject.toml`) |
| Code described | commit `d155d48` on branch `main` |

A self-assessment by the team, not an independent audit. It covers the application and its container setup at the commit named above; the server that hosts the public demo is out of scope. Statements about the code point to the file or test that supports them; statements about the deployment come from the branch `ops/deploy-neural-ledger`. This is engineering evidence, not legal advice.

# Contents

**1 Executive summary** **2**

**2 Scope and method** **2**

**3 System and trust boundaries** **3**

**4 Security tiers** **5**
4.1 Transport . . . . . 5
4.2 Application and authentication . . . . . 5
4.3 Data at rest . . . . . 6
4.4 Key management . . . . . 7
4.5 Host . . . . . 7

**5 Account deletion and the audit trail** **8**

**6 Controls and evidence** **9**

**7 Known-risk register** **11**

**8 Residual risk after option B** **13**

**9 Conclusion** **14**

# 1 Executive summary

Neural Ledger is a cash-flow copilot that reads bank statements, forecasts a user's liquidity and can create a savings rule in a *sandbox* bank. Its core security property is that no language model can move money or set a number: amounts are computed by deterministic code, actions need a signed authorization that only the user can create, and every reply is checked against the facts the tools returned. That property is covered by attack tests S01–S24, S30–S32 and S35–S38, and by the memory tests, and was not changed by this assessment.

This assessment was triggered by a review of the public demo. Anyone can sign up there, so a visitor could store a real bank statement on a server that is meant for synthetic data. Personal data at rest was only partly encrypted, and the encryption key sat on the same volume as the database. The team chose *option B*: keep sign-up open, harden the application and say so (ADR-038). After option B:

- sign-up shows a sandbox notice that cannot be dismissed, with an acknowledgement the server enforces and records;
- every personal payload in the database is sealed with AES-256-GCM, and the email is found through a keyed HMAC index;
- in deployed mode the key comes from a Docker secret, and the API refuses to start if the key is on the data volume;
- users can delete their account and all of its data, and the shared audit chain still verifies afterwards.

The largest residual risks (Section 8) are four. A notice cannot stop a visitor from uploading real data (R01, R02). Anyone with root on the host can read both the key and the data (R03). Until the server is migrated, the demo still runs the old configuration (R24). Finally, consent and opposition are not built (R23).

**Version 1.1** reviewed four features merged since (ADR-039, ADR-040, ADR-044, ADR-047). They bring new third-party text, a provider token with full access, and outbound calls to addresses users choose. The agent stays read-only towards all of them, and every new store is sealed and erased with the account. The new risks were R25–R31. The largest is that a connected server may lie about a tool being read-only (R25).

**Version 1.2** reviews what `main` gained after that: the fixes of pull request #58, which now count live connectors as live mode (R28) and give connection calls their own rate budget (R29); a download of the user's own data (G02, closed); and typed user memory, saved only (ADR-048). Memory adds a place where text the user types becomes stored state, and the answer is that nothing the model says can write it: the two memory tools take no arguments, the user confirms every declaration, and a per-user fence stops a reply from outliving a revoked consent. The new risks are R32–R35. None is rated above medium likelihood, and the largest impact belongs to the download, which anyone holding a stolen session cookie could use (R35).

# 2 Scope and method

**In scope**

The backend (`backend/src/neural_ledger`), the web client (`web/`), the contracts, the container setup (`docker-compose.yml`, `docker-compose.data-key.yml`, `backend/Dockerfile`) and the documentation in `docs/`, at commit `d155d48`.

**Out of scope**
The host that runs the public demo (operating system, SSH, Docker daemon, backups), Cloudflare's configuration, and third-party providers. The deployment override and runbook were read on `ops/deploy-neural-ledger` but not tested against the server.

**Method**
Code review of the identity, persistence, cryptography and erasure paths. The full test suite was run: `python -m pytest -m ""` gave 1556 passed, 1 skipped, 9 expected failures (documented gaps) and 2 unexpected passes (non-strict property tests that existed before); `ruff` was clean; the web client passed 176 tests, lint, typecheck and build. The redeploy was also rehearsed with Docker on a volume written by the previous code:

1. before the migration, the new image refused to start in deployed mode;
2. a dry run of the migration changed nothing; the real run re-keyed and sealed 14 values and upgraded the schema;
3. after the old key file was deleted, the API started in deployed mode, and sign-in by email worked;
4. the raw database held neither the email nor the name;
5. an account deletion kept the audit chain valid;
6. a wrong key was refused.

**Update 1.1**
A code review of receipts, concept cards, Fintual and connections at staging `d959a04`, whose CI run passed: 2366 backend tests passed, 9 skipped, 7 expected failures and 1 unexpected pass; `ruff` clean; 205 web tests passed. The suitability suite has no expected failures left. Both concept-card indexes reach recall@3 of 1.0 on 48 questions.

**Update 1.2**
A code review of the data export, user memory and the connector fixes, at commit `d155d48` on `main`, whose CI run passed: 2980 backend tests passed, 9 skipped, 6 expected failures and 1 unexpected pass; `ruff` clean; 235 web tests passed. The compliance map has 27 tests: 24 pass and 3 are expected failures (G01, G05, G06). The memory design was read from its ADRs and its code; the calculation that would let a declared salary change the forecast is written but not called, so it was not assessed as a control.

**Limits**
No penetration test of the running demo, no dependency vulnerability scan, and no review by counsel. Live connectors, Fintoc and a real Fintual link were not exercised against the real services.

## 3 System and trust boundaries

Figure 1 shows the public demo. Visitors reach Cloudflare's edge over TLS. A tunnel opened by the host carries requests to the web container (nginx), which proxies `/api` to the API container. The API holds the data key in memory, reads it from a secret mounted on its own, and keeps its data in SQLite on a Docker volume. In live mode it calls model providers; the public demo runs offline, and `/health` reports `"mode": "offline"`.

Figure 2 follows a statement upload, the path that carries the most sensitive data. The raw file exists only in memory during the request. Redaction runs before anything is stored, and every payload is sealed before it reaches SQLite.

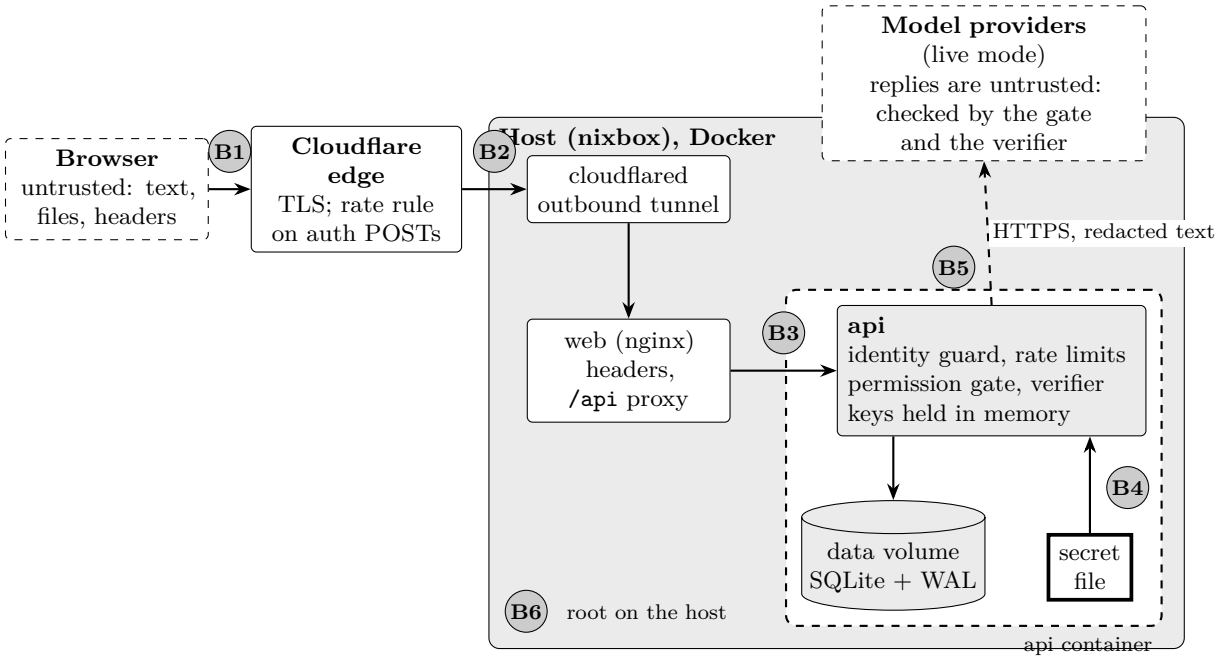



**Figure 1:** Trust boundaries. B1 visitors to edge; B2 tunnel into the host; B3 the API's perimeter; B4 the secret's mount; B5 model providers (live mode only); B6 the host itself, out of scope.

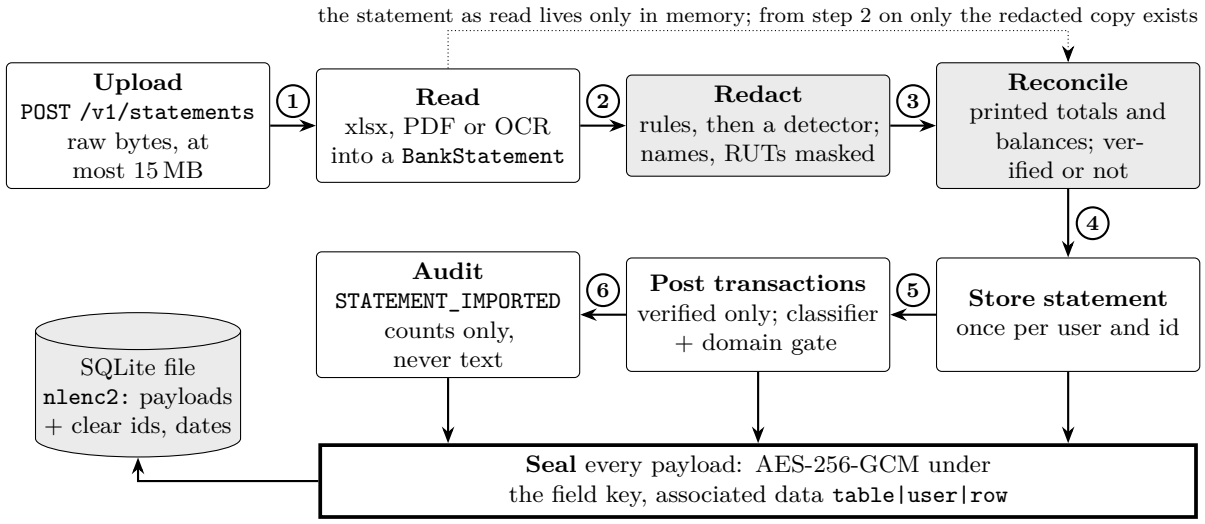



**Figure 2:** Statement upload: read, redact, reconcile, then seal and store.

# 4 Security tiers

Table 1 summarizes what each layer protects; the subsections give the detail and the evidence.

| Tier | Protected by | Not protected |
|---|---|---|
| Transport | TLS at Cloudflare's edge; HTTP to HTTPS redirect, HSTS and a strict CSP in nginx; no host port published (deployment branch); `Secure` cookies | The hop inside the host is plain HTTP on the Compose network; port 8000 is published in the base compose file |
| Application and authentication | Cookie sessions (12 h, `HttpOnly`, `SameSite=Strict`); scrypt; identity header rejected; Origin check; rate limits; security headers; upload limits and type checks; permission tiers, signed authorizations, response verifier; sandbox notice | No MFA, lockout, email verification or password recovery; sign-up stays open |
| Data at rest | AES-256-GCM on every personal payload with per-row associated data; email as an HMAC blind index; audit records sealed; `secure_delete`; VACUUM after migration; account deletion | Ids, owner ids, session ids, posting dates and counts stay readable |
| Key management | HKDF subkeys per purpose; key from a Docker secret or the environment; deployed mode refuses a missing key, a key on the data volume, the old key file and the development HMAC secret; key check before any write; idempotent migration and rotation | No KMS or HSM; the key is a file on the host; manual rotation and backup |
| Host | Out of scope of this change | SSH hardening pending; root reads the key and the data |

**Table 1:** Security tiers.

## 4.1 Transport

TLS ends at Cloudflare's edge. The deployment override on `ops/deploy-neural-ledger` publishes no host port, so the API can be reached only through the tunnel. Its nginx configuration redirects plain HTTP to HTTPS and adds HSTS, `X-Frame-Options: DENY`, a referrer policy and a strict content security policy. The API itself adds `nosniff` and `no-store` to every response (G09). Behind the tunnel, nginx and the API talk plain HTTP on the Compose network, and the API trusts `X-Forwarded-For` only from private ranges. The base `docker-compose.yml` publishes port 8000 for development, so a deployment must use the override.

## 4.2 Application and authentication

- **Identity** (accounts mode, `adapters/inbound/http/identity.py`): the user id comes only from a session cookie resolved on the server. `X-User-Id` is answered with 400. An unsafe request needs an allowed Origin, or a JSON body when there is no Origin. Every non-public route needs a session; a route-inventory test (`tests/http/test_auth_api.py`) fails if a new route is public by accident (S16).
- **Sessions**: twelve hours from creation, no sliding, revoked at sign-out. Only the token's SHA-256 is stored (S17).
- **Passwords**: scrypt with $N = 2^{14}$, $r = 8$, $p = 1$. Sign-in costs the same and answers the same whether or not the email exists (S20); sign-up reveals a registered email with 409.

- **Rate limits** (G07): per caller and per IP, one-minute windows. Ten sign-in or sign-up attempts per IP; a request with no session and no identity header is counted by its IP, so one address using its budget up never locks out another. Five account deletions per caller, ten uploads per caller, and ten per caller on investment syncs and on each connection call. Counters live in memory, per process.
- **Input**: uploads are typed by their bytes and capped at 15 MB (C05); hand-parsed JSON bodies are capped at 16 KiB; SQL is always parameterized, and a static test checks every SQLite module (C08).
- **Agent**: four permission tiers and a deterministic gate; signed, single-use authorizations only the user can create; terms re-read from storage; a verifier that rejects unsupported numbers and missing warnings (S01–S14, S19, S22, S23). Receipts, concept cards, Fintual goal names and connected services' answers reach the model only as untrusted text, and no tool stores, links or syncs anything (S24, S30–S32, S36). The memory tools take no arguments and refuse any; twelve tool names that would write memory are T3.
- **Outbound calls**: a connected server's address must be HTTPS and resolve only to public addresses; redirects are never followed; answers are capped at 1 MB after 10 s. A tool is enabled only if it is read-only, not destructive, takes no arguments and has no write verb in its name (S35, S37).
- **Email domains**: sign-up refuses disposable-inbox domains and, with `dns`, domains that receive no mail.
- **Sandbox notice**: sign-up requires `sandbox_acknowledged = true`, and `USER_REGISTERED` records `sandbox_notice=sandbox-1`. The upload screen warns again.

### 4.3 Data at rest

`adapters/outbound/persistence/field_cipher.py` seals each payload with AES-256-GCM, a fresh 12-byte nonce, and associated data `<table>|<user_id>|<row id>`. A value moved to another row or table does not open, and a value stored in clear is refused, so a field cannot be downgraded to plain text. Table 2 lists what is sealed.

| Data | Sealed content | Still readable |
|---|---|---|
| Statements, transactions | lines, descriptions, merchants, amounts, balances, categories | ids, owner, posting date |
| Receipts | store, date, total, folio, category, issues | ids, owner |
| Investment links, snapshots, documents | provider token and email, goals and values, funds, contributions | ids, owner |
| Connections and their results | server address, access token, enabled tools, redacted answers | ids, owner, tool name |
| Memory (consent, declared salary versions, proposals) | the consent and counters, the amounts and pay months, the origin references, the proposals | ids, owner, chain id, version, type |
| Accounts | email, password hash, creation time | HMAC email index |
| Profiles | name, language, questionnaire answers, tolerance | owner |
| Proposals, authorizations, rules | terms, amounts, signatures, status | ids, owner, session id |
| Audit records | the whole event, hash chain included | sequence, event id, owner, session id |
| Sessions | none (no personal content) | reference, owner, token digest, times |

**Table 2:** What a copy of the database file reveals.

Evidence: `tests/adapters/test_encryption_at_rest.py` writes an account, its questionnaire, a statement, a proposal, an authorization and a rule over HTTP, then searches the raw database and WAL bytes for each known value (none is found). It also upgrades a hand-built schema 0 file and checks that the old email disappears from the file. S15, C01 and C02 check that redaction keeps names, RUTs and card numbers out of every surface, the database included.

## 4.4 Key management

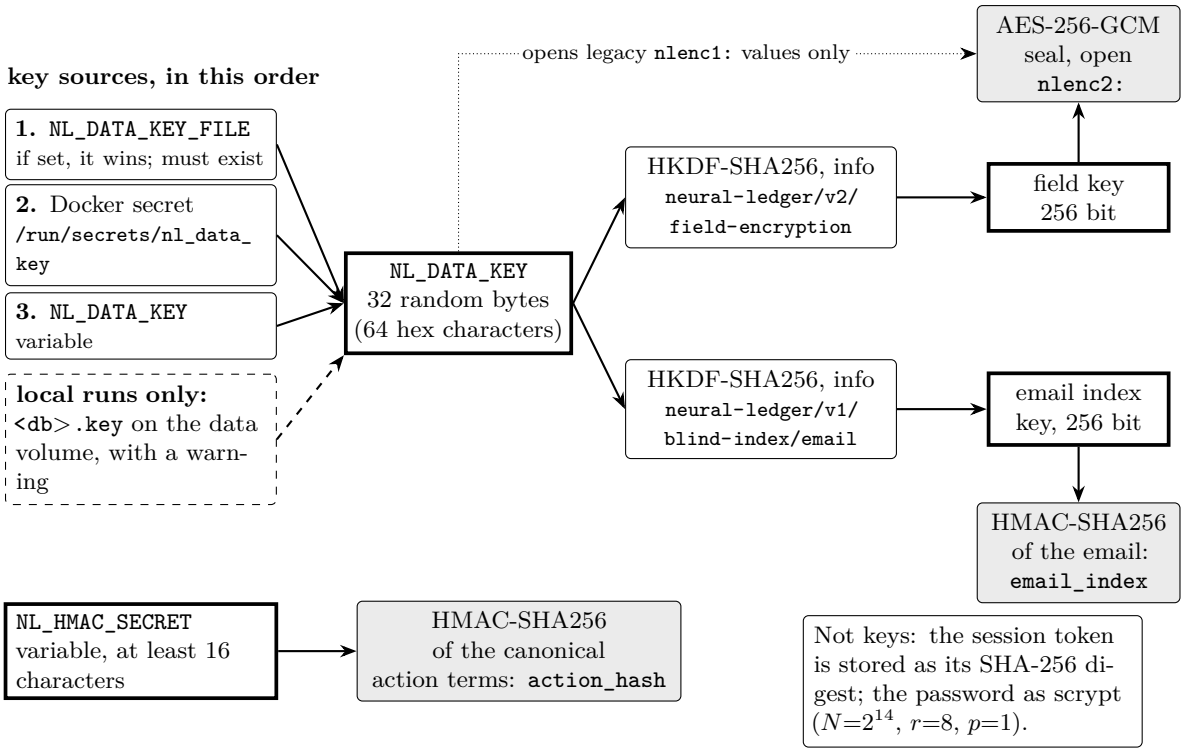



**Figure 3:** Key hierarchy. Each purpose has its own subkey derived with HKDF-SHA256; the HMAC secret for authorizations is a separate root.

The data key is read from `NL_DATA_KEY_FILE` or the Docker secret `/run/secrets/nl_data_key`, then from `NL_DATA_KEY` (`bootstrap/data_key.py`, Figure 3). With `NL_DEPLOYMENT=deployed` the API refuses to start in four cases, checked before it opens the database:

- there is no key;
- the key file shares the database's mount (read from `/proc/self/mountinfo`) or its directory;
- the old `<db>.key` still exists;
- the HMAC secret is the development default.

A sealed key check in the `meta` table rejects a wrong key before anything is written. `neural-ledger migrate-data` re-seals every value in one transaction, with a dry run, and is idempotent; the same command rotates keys. Local runs keep a key file next to the database and print a warning. `tests/adapters/test_data_key.py` covers each refusal, the wrong key, the dry run, idempotence, and a migration stopped by a value that opens with no given key.

## 4.5 Host

The host is out of scope. Root on the host can read the mounted secret, the volume and the API's memory; so can anyone who controls the Docker daemon. SSH hardening on the server is pending.

Backups of the volume must never contain the secret file, and the key needs a backup of its own, outside the database backups.

## 5 Account deletion and the audit trail

The audit trail is evidence, so it must resist editing. It is also personal data, so a user must be able to have it erased (Ley 21.719 art. 7). Option B reconciles the two with tombstones. When an account is deleted, each of its audit events keeps its place in the chain (sequence, `prev_hash`, `event_hash`) and loses everything else. One `ACCOUNT_DELETED` event, with no user id, commits to the exact list of erased sequences. `verify_chain` accepts a tombstone only under such a commitment (Figure 5). The deletion itself is one transaction (Figure 4).

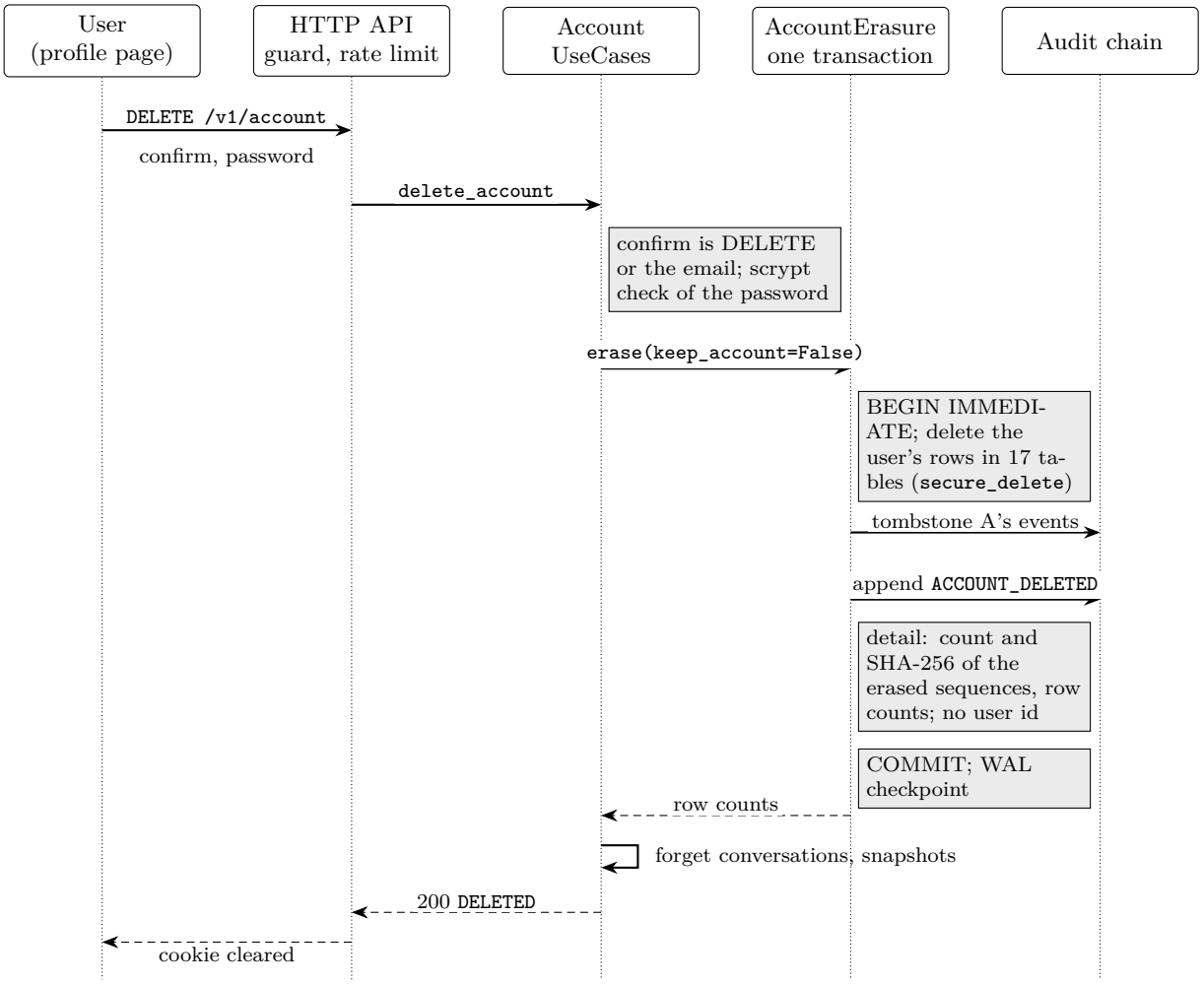
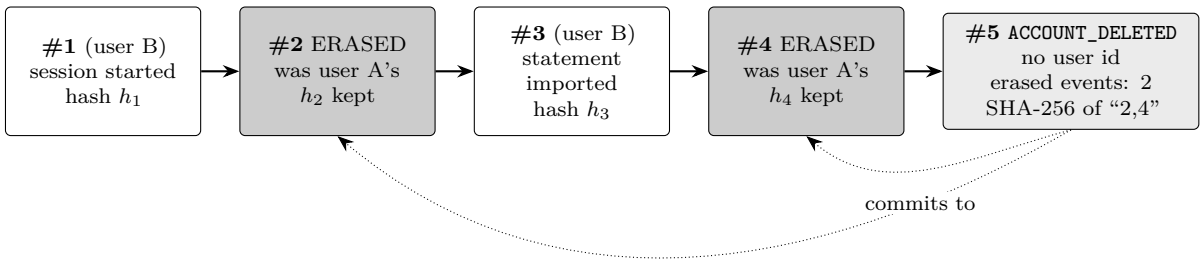



**Figure 4:** Account deletion: password and confirmation, one transaction, tombstones, a committing erasure event.

Evidence (`tests/security/test_account_deletion.py`, run on the memory and SQLite adapters):

- A wrong password or confirmation deletes nothing.
- Deleting user A ends both of A's sessions and leaves user B's statements, transactions, profile, rules and sign-in intact.
- No row of any SQLite table refers to A's id, email, name or sessions, including the contents of opened sealed values; A's id is not in the raw file at all.




Arrows are the *prev_hash* links. The links and hashes of #2 and #4 are unchanged, so #3 and #5 still verify. A tombstone is accepted only if a later erasure event commits to exactly the set of tombstones that name it.

**Figure 5:** The chain after an erasure.

- A can register again with the same email.
- Forged tombstones fail verification: erasing B's event, a tombstone without an erasure event, an edited count, and a tombstone after its erasure event.

Demo personas are reset, not deleted, and the reset is recorded as `DEMO_DATA_RESET`.

## 6 Controls and evidence

The compliance map in `tests/security/test_compliance.py` holds 27 tests. Built controls C01–C13, G02–G04 and G07–G12 are plain tests. G01 (demo mode has no authentication), G05 (opposition and blocking) and G06 (consent) are strict expected failures, so CI stays green and the gaps stay visible.

| Control | Behaviour verified | Evidence |
|---|---|---|
| Gate and tiers | A hostile model cannot transfer, add a beneficiary, confirm, or raise its tier; T3 names blocked and audited | S01–S06, S13, S19, S23 |
| Authorizations | Over-amount, expired, replayed (8 concurrent), tampered terms blocked | S03–S05, S11 |
| Verifier | Invented numbers, contradictions and false success claims replaced | S07, S08, C12 |
| Suitability | Plans capped at min(tolerance, capacity); hard rules H1–H5 | S09, `tests/suitability` |
| Identity | Header rejected, no session 401, cross-account reads empty, demo personas allowlisted | S16–S18, C04 |
| Sign-in | No enumeration, same scrypt cost | S20 |
| Redaction | No name, RUT, email, phone or card number on any surface | S15, C01–C03, C13 |
| Rate limits | 429 over budget, before any use case | G07 |
| Sandbox notice | Sign-up refused without acknowledgement; version recorded | `tests/http/test_auth_api.py` |
| Encryption at rest | No known value in the raw file; owner binding; no plain-text downgrade | G11, `test_encryption_at_rest.py`, `test_field_cipher.py` |
| Key management | Deployed refusals, wrong key, migration | `test_data_key.py` |
| Deletion | B intact, nothing of A left, chain verifies, forged tombstones fail | G03, `test_account_deletion.py` |
| Audit integrity | Edited event detected | C10, `test_audit_chain_detects_tampering` |
| Secrets in the repository | No `.env`, API key, card number or real statement tracked | C07 |
| Concept cards | Injected card text reaches no T1 or T2 tool; numbers outside the card and a missing `EDUCATIONAL_NOT_ADVICE` fail verification | S24 |
| Investments | Goal names untrusted and never narrated; credentials never leak; link and unlink T3; only verified documents kept; personas cannot import | S30–S32 |
| Connections | False read-only labels, injected answers, SSRF targets and token hunts | S35–S38, `test_mcp_http.py` |
| Suitability report | Plans state the permitted profile and why; superseded plans refused; off-track goals warned | UG01–UG03, `tests/suitability` |
| Rate limits on outside calls | Each `POST /v1/connections*` has its own budget: 10 per caller, 30 per IP | `test_every_call_to_an_outside_service_has_its_own_budget` |
| Live mode reporting | Live connectors make the mode live and `/health` reports `connectors` | `tests/security/test_connectors.py` |
| Data export | One file with what a deletion would erase, proposals and authorizations included (only the latest investment snapshot); every erased table mapped to a section; never a hash, token, secret or signature; audited | G02, `tests/security/test_data_export.py` |
| Memory | No tool writes; only the user confirms; consent gates every read; stale entries never reach a model; an import cannot outlive a deleted account | `test_memory_agent.py`, `test_import_after_deletion.py`, `test_memory_dependency.py`, `test_memory_acceptance.py` |

**Table 3:** Controls and the tests that evidence them (`docs/SECURITY.md`, `backend/tests`).

# 7 Known-risk register

Likelihood and impact use three levels. High likelihood means expected on the public demo without further action; Low means it needs an unusual mistake or a privileged attacker. High impact means real personal data exposed or lost, or the security model broken; Low means limited to metadata, availability of the demo, or the sandbox. Owners are the team's roles (A finance, B agent, C platform, D frontend, E quality) or *Ops*, the people who run the server. The register is our own analysis of the code and of the option B change.

**Table 4:** Known-risk register (L likelihood, I impact).

| ID | Risk | L | I | Current mitigation | Status, owner |
|---|---|---|---|---|---|
| R01 | A visitor uploads a real statement or real personal data to an account | M | H | Notice and acknowledgement; redaction at import; sealed at rest; self-service deletion | Accepted for the demo; C, E |
| R02 | A real statement uploaded under a shared persona is shown to every visitor of it | M | H | Upload warning; any visitor can reset the persona | Open; C, D: refuse uploads in persona sessions |
| R03 | Root on the host reads the mounted key, the volume and process memory | L | H | None in the application; host out of scope | Open; Ops: SSH hardening pending |
| R04 | The data key is lost, and every account and statement with it | L | H | Documented backup apart from the database | Open; Ops |
| R05 | The key is backed up with the database, or stored in `app/.env` or the repository | M | H | Secret file outside the volume; `deploy/secrets` ignores keys; docs | Open; Ops |
| R06 | The pre-migration volume backup holds the old key next to the old data | M | H | Redeploy step 11: delete it once verified | Pending redeploy; Ops |
| R07 | A copy of the database shows ids, counts, session ids and posting dates | H | L | Content sealed; metadata kept for lookups | Accepted; C |
| R08 | After a deletion, tombstones keep positions, event ids and hashes, and exports made earlier keep events in clear | M | M | Content erased; erasure event has no user id; exports kept in the incident folder only | Accepted; C, E |
| R09 | Account takeover by password guessing (no lockout, no MFA) | M | M | scrypt; 10 attempts per minute per IP; Cloudflare rule on auth routes | Open; C |
| R10 | A stolen session cookie works for up to 12 hours | L | M | HttpOnly, SameSite=Strict, Secure, CSP; deletion still needs the password | Open; C |
| R11 | Sign-up reveals whether an email is registered (409) | H | L | Sign-in does not; rate limits | Accepted; C |
| R12 | Abuse of open sign-up: mass accounts, storage, statement parsing load | M | M | Per-IP limits; 15 MB uploads; Cloudflare rule | Open; C, Ops |
| R13 | Prompt injection through chat or merchant text steers the model's prose | H | M | Tiers, gate, untrusted wrapping, verifier; no money moves without the user | Mitigated; B |
| R14 | Misleading advice the verifier cannot catch (semantic contradictions; estimator warnings dropped by `LedgerModelSource`) | M | M | Facts and warnings checked; narration fallback | Open; A, B |

| ID | Risk | L | I | Current mitigation | Status, owner |
|---|---|---|---|---|---|
| R15 | Redaction misses a name outside the transfer patterns; with Jev on, it leaves the country | M | M | Rules plus optional detector; sealed at rest; Jev off by default | Open; C |
| R16 | Live providers receive data abroad without documented safeguards (art. 27) | L | M | Offline by default; `/health` shows the mode | Open; E (counsel) |
| R17 | A misconfiguration exposes demo identity or live mode publicly | L | H | Compose fixes accounts mode; deployed mode refusals; `/health` | Mitigated; Ops |
| R18 | Any visitor resets a shared persona, erasing other visitors' demo data | M | L | Reset is audited; personas hold synthetic data | Accepted; C |
| R19 | Someone with write access and the key forges tombstones and their commitment | L | L | Commitment check; unkeyed chain can be recomputed by such an attacker | Accepted; C |
| R20 | `/health` opens every audit record on each poll; cost grows with the log | M | L | Demo-sized log | Open; C: cache the chain check |
| R21 | Supply chain: dependency ranges (>=) and base images not pinned by digest | M | L | CI runs the full suite on each change | Open; C |
| R22 | A request in flight during a deletion writes rows for the deleted user | L | M | Sessions deleted in the same transaction; the next request is 401 | Open; C |
| R23 | Compliance gaps: no opposition or blocking (G05), consent (G06), impact assessment | M | M | Gaps tested as strict expected failures and documented | Open; C, E |
| R24 | The demo still runs the pre-option-B setup (key on the volume, plain fields) until the server is migrated; the deployment files are on an unmerged branch | H | H | Redeploy steps written and rehearsed | Pending redeploy; Ops |
| R25 | A connected MCP server lies: a tool labelled read-only and argument-free moves money or changes the user's account there | M | H | Only the user's sync calls tools, with empty arguments; write verbs refused; labels re-checked each sync; off in deployed mode | Accepted with a warning to users; C |
| R26 | DNS rebinding: an address that resolves to a public IP when checked and to a private one when the client connects | L | M | Public-address check, no redirects, 10 s, 1 MB; marked `shortcut:` in the adapter | Open; C: pin the checked address in the connection |
| R27 | A stolen Fintual token reads the whole account: the API has no read-only scope and no revocation | L | H | Sealed and bound to user and adapter; never returned, logged or prompted; deployed mode refuses `fintual` | Open; C |
| R28 | `NL_CONNECTORS=live` sends tokens to user-chosen servers and to Fintoc, but `/health` still reports `offline` | L | M | Deployed mode keeps connectors off unless `NL_CONNECTORS_LIVE` is set | Fixed in PR #58 (merged): live connectors make the mode live, `/health` reports them; C |

| ID | Risk | L | I | Current mitigation | Status, owner |
|---|---|---|---|---|---|
| R29 | Connection syncs are limited only by the general 300 per minute budget, and each one calls an outside server | L | L | 10 s timeout and 1 MB cap per call | Fixed in PR #58 (merged): 10 per caller, 30 per IP; C |
| R30 | A concept card states a wrong definition; the verifier checks numbers, not meaning | L | M | Team-written cards with cited sources; card tests; eval gate; `EDUCATIONAL_NOT_ADVICE` warning | Open; B: review cards |
| R31 | The first Fintoc sync has no previous balance, so it cannot be reconciled | H | L | Stored unverified (`NO_SUMMARY`) and not posted; later syncs reconcile | Accepted; A |
| R32 | The memory fence orders one API process only, and a dependent action proposal created before a memory change was seen stays pending until its 30 minutes end | L | L | Counters are compared on every read, so a change from another process is still caught; a proposal's terms never come from memory; one process is how the API runs | Accepted; C |
| R33 | Memory routes are live in any accounts-mode deployment while ADR-048 and ADR-049 are still Proposed and unsigned by Roles A, C and E | M | L | Saved only: no number, limit or verdict changes; consent is off until the user grants it; shared personas keep none | Open; A, C, E: sign or switch off before real users |
| R34 | An older binary opens a database that holds memory and its deletion or persona reset leaves the memory behind (no schema version bump) | L | M | Documented as "no downgrade" in `docs/SECURITY.md`; the tables are additive | Open; C: bump the version with the next table |
| R35 | A stolen session cookie downloads the whole copy of the user's data in one request | L | H | HttpOnly, SameSite=Strict, Secure, 12 hours; `no-store`; audited as `DATA_EXPORTED`; never the password hash or a secret | Open; C: ask for the password again, as deletion does |

# 8 Residual risk after option B

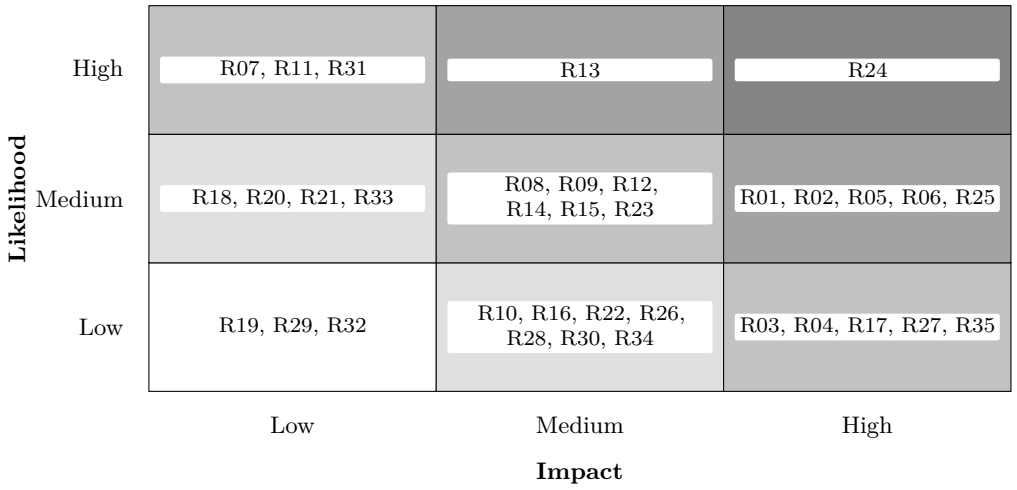



**Figure 6:** Residual risks by likelihood and impact. Darker cells need attention first.

Option B moves three risks down. A copy of the database or of the volume no longer reveals content; before, the email, profiles, proposals, rules and the audit chain were plain text, and the

key sat next to the data. A user can now remove their data. And the sandbox rule is explicit and recorded. What remains, in order of priority:

1. **Complete the redeploy** (R24, R06): migrate the server with the rehearsed steps, delete the old key file and the pre-migration backup, and merge the deployment branch so that the repository describes what runs.
2. **Real data despite the notice** (R01, R02). This is the risk option B accepts. Refusing statement uploads in persona sessions would remove R02 at little cost. Option A (closing sign-up, or Cloudflare Access) remains available if real data is ever found.
3. **The host** (R03, R05). Harden SSH, restrict who controls Docker, and keep the key and its backup out of the database backups.
4. **Authentication depth** (R09, R10, R12). Add a per-account lockout or a delay, email verification, and an abuse budget for sign-up.
5. **Compliance** (R23, R16). Export is built (G02). Build consent and opposition, write the impact assessment, and document transfer safeguards before any use with real users.
6. **Connections** (R25, R26). Pull request #58 closed R28 and R29. Pin the address that was checked, and keep live connectors off the public demo.
7. **Memory and the export** (R33, R34, R35). Roles A, C and E sign ADR-048 and ADR-049 or switch the routes off before real users; bump the schema version when the next table is added; and ask for the password before the copy is downloaded. If the gate on 22 October 2026 lets a declared salary change the forecast, the assessment of memory must be redone, because that is the point at which words become numbers.

# 9 Conclusion

For a hackathon sandbox, the application's controls are proportionate. Its core claim, that a model cannot move money or invent a number, is tested against a hostile model. After option B, personal data at rest is sealed with a key kept apart from the data, and it can be erased without breaking the evidence trail. The remaining exposure is mostly organizational: what visitors choose to upload, how the server is operated, and the legal duties that a production service would still have to meet.